

Writeup: Challenge Crackme essai_01

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_06_essai01OK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `M@nouK@nA$$te@m*`

Résumé

Writeup Challenge Crackme essai_01

Informations initiales

Avant de commencer l'analyse, j'ai récolté des informations de base sur ce fichier.

```
file essai_01
```

essai_01: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, not stripped

Le binaire est un exécutable ELF 64-bit pour architecture x86-64. Il est statiquement lié (toutes les bibliothèques sont incluses directement dans l'exécutable) et n'est pas "strippé", ce qui signifie que les symboles de débogage sont toujours présent

Outils Utilisés

Analyse des données

J'ai examiné la section des données pour comprendre l'algorithme de vérification :

```
objdump -s -j .data essai_01
```

essai_01: format de fichier elf64-x86-64

Contenu de la section .data :

```
402000 0f022c2d 3709022c 03666636 27022f68  ..,-7.,.ff6'./h
402010 050c0009 030f0702 0b040e08 010a060d  .....
402020 42476f6f 64204a6f 62210a00 42616420  BGood Job!..Bad
402030 50617373 776f7264 210a0000 00000000  Password!.....
402040 00000000 00000000 00000000 00000000  .....
```

J'ai identifié trois parties importantes dans la section de données :

Une comparaison indirecte avec les valeurs attendues

Les outils utilisés pour ce challenge étaient :

file (identification du type de fichier)
strings (extraction des chaînes de caractères)
nm (examen des symboles)
objdump (désassemblage du code)
readelf (analyse des sections ELF)
un script JavaScript pour inverser l'algorithme

Analyse Statique

Analyse statique

Extraction des chaînes de caractères

J'ai commencé par extraire les chaînes de caractères présentes dans le binaire :

```
$ strings essai_01
% @
,-7
ff6'
BGood Job!
Bad Password!
essai_01.s
data_table
sequence
magic_byte
flag_length
good_msg
bad_msg
buffer
index
count_loop
end_count
compare_loop
correct_password
wrong_password
__bss_start
_edata
_end
```

Les chaînes "Good Job!" et "Bad Password!" suggèrent que le programme attend un mot de passe ou un flag à vérifier.

Examen des symboles

Comme le binaire n'est pas strippé, j'ai pu examiner les symboles présents :

```
$ nm essai_01
000000000040202c d bad_msg
00000000004020a0 D __bss_start
```

```
000000000040203b d buffer
000000000040104b t compare_loop
000000000040107f t correct_password
000000000040101e t count_loop
0000000000402000 d data_table
00000000004020a0 D _edata
00000000004020a0 D _end
0000000000401031 t end_count
0000000000000010 a flag_length
0000000000402021 d good_msg
000000000040209f d index
0000000000402020 d magic_byte
0000000000402010 d sequence
0000000000401000 T _start
00000000004010a4 t wrong_password
```

L'analyse des symboles révèle plusieurs points intéressants :

flag_length avec la valeur 0x10 (16 en décimal), il semble que le programme attend un mot de passe de 16 caractères

Des sections comme compare_loop, correct_password et wrong_password qui indiquent la logique de vérification

Des variables comme data_table, sequence et magic_byte qui sont probablement utilisées dans l'algorithme de vérification

Analyse des sections

```
readelf -S essai_01
```

Il y a 6 en-têtes de section, débutant à l'adresse de décalage 0x2340 :

Analyse approfondie

Désassemblage du code

```
objdump -d essai_01
```

essai_01: format de fichier elf64-x86-64

Désassemblage de la section .text :

```
0000000000401000 <_start>:
401000:  b8 00 00 00 00      mov    $0x0,%eax
401005:  bf 00 00 00 00      mov    $0x0,%edi
40100a:  48 be 3b 20 40 00 00  movabs $0x40203b,%rsi
401011:  00 00 00
401014:  ba 64 00 00 00      mov    $0x64,%edx
401019:  0f 05              syscall
40101b:  48 31 c9            xor    %rcx,%rcx
000000000040101e <count_loop>:
40101e:  8a 81 3b 20 40 00    mov    0x40203b(%rcx),%al
```

```

401024:  3c 0a          cmp  $0xa,%al
401026:  74 09          je   401031 <end_count>
401028:  3c 00          cmp  $0x0,%al
40102a:  74 05          je   401031 <end_count>
40102c:  48 ff c1       inc  %rcx
40102f:  eb ed          jmp  40101e <count_loop>
0000000000401031 <end_count>:
401031:  c6 81 3b 20 40 00 00  movb $0x0,0x40203b(%rcx)
401038:  48 83 f9 10     cmp  $0x10,%rcx
40103c:  75 66          jne  4010a4 <wrong_password>
40103e:  b9 10 00 00 00  mov  $0x10,%ecx
401043:  c6 04 25 9f 20 40 00  movb $0x0,0x40209f
40104a:  00
000000000040104b <compare_loop>:
40104b:  48 0f b6 14 25 9f 20  movzbq 0x40209f,%rdx
401052:  40 00
401054:  48 0f b6 9a 10 20 40  movzbq 0x402010(%rdx),%rbx
40105b:  00
40105c:  8a 83 3b 20 40 00  mov  0x40203b(%rbx),%al
401062:  32 04 25 20 20 40 00  xor  0x402020,%al
401069:  8a 9b 00 20 40 00  mov  0x402000(%rbx),%bl
40106f:  38 d8          cmp  %bl,%al
401071:  75 31          jne  4010a4 <wrong_password>
401073:  fe 04 25 9f 20 40 00  incb 0x40209f
40107a:  48 ff c9       dec  %rcx
40107d:  75 cc          jne  40104b <compare_loop>
000000000040107f <correct_password>:
40107f:  b8 01 00 00 00  mov  $0x1,%eax
401084:  bf 01 00 00 00  mov  $0x1,%edi
401089:  48 be 21 20 40 00 00  movabs $0x402021,%rsi
401090:  00 00 00
401093:  ba 0a 00 00 00  mov  $0xa,%edx
401098:  0f 05          syscall
40109a:  b8 3c 00 00 00  mov  $0x3c,%eax
40109f:  48 31 ff       xor  %rdi,%rdi
4010a2:  0f 05          syscall

```

En analysant le code assembleur, j'ai pu identifier plusieurs sections importantes :

Lecture de l'entrée utilisateur (syscall à 0x401019)

Vérification de la longueur du mot de passe (0x401038 - compare avec 0x10 = 16)

Boucle de comparaison qui utilise un mécanisme de vérification spécial
(0x40104b - 0x40107d)

Analyse Dynamique

L'exécution de ce script m'a donné le mot de passe correct :

M@nouK@nA\$\$te@m*

Vérification

Identification du Mécanisme de Validation

À 0x402020 : Un octet magique magic_byte (0x42, qui est 'B') utilisé dans l'algorithme

Comprendre l'algorithme

En analysant le code assembleur et les données, j'ai pu déterminer que l'algorithme de vérification fonctionne comme suit :

Le programme lit 16 caractères d'entrée

Pour chaque position (0-15), il utilise le tableau sequence pour déterminer quel caractère vérifier

Le caractère est XOR avec magic_byte (0x42)

Vérification

J'ai vérifié le mot de passe en l'entrant dans le programme :

```
./essai_01  
M@nouK@nA$$$te@m*
```

Good Job!

Découverte du Flag

Le résultat est comparé à la valeur correspondante dans data_table

Solution

Pour trouver le mot de passe correct, j'ai écrit un petit script qui inverse l'algorithme :

```
javascript// Les tableaux pour la vérification  
const data_table = [0x0f, 0x02, 0x2c, 0x2d, 0x37, 0x09, 0x02, 0x2c, 0x03, 0x66, 0x66, 0x36, 0x27, 0x02, 0x2f, 0x68];  
const sequence = [0x05, 0x0c, 0x00, 0x09, 0x03, 0x0f, 0x07, 0x02, 0x0b, 0x04, 0x0e, 0x08, 0x01, 0x0a, 0x06, 0x0d];  
const magic_byte = 0x42;
```

// Tableau pour stocker le mot de passe

```
const password = new Array(16).fill(0);
```

// On decode le mdp

```
for (let i = 0; i < 16; i++) {
```

```
    const pos = sequence[i]; //Position dans ltableau password  
    password[pos] = data_table[pos] ^ magic_byte; //XOR avec le magic_byte  
}
```

//Convertir en caractères ASCII et affichage

```
console.log("Mot de passe : " + String.fromCharCode(...password));
```

L'exécution de ce script m'a donné le mot de passe correct :
M@nouK@nA\$\$te@m*

Conclusion

Conclusion

Ce challenge, il était intéressant car il utilisait plusieurs techniques pour obfusquer le mot de passe :

Une permutation de l'ordre des caractères grâce au tableau sequence

Une opération XOR avec un octet magique (0x42)

En-têtes de section

En-têtes de section :

[Nr]	Nom	Type	Adresse	Décalage
	Taille	TaillEntrée	Fanion Lien Info	Alignement
[0]	NULL	0000000000000000	0000000000000000	0 0 0
[1]	.text	PROGBITS	0000000000401000	00001000
	0000000000000000cb	0000000000000000	AX	0 0 16
[2]	.data	PROGBITS	0000000000402000	00002000
	0000000000000000a0	0000000000000000	WA	0 0 4
[3]	.symtab	SYMTAB	0000000000000000	000020a0
	00000000000000001c8	000000000000000018		4 15 8
[4]	.strtab	STRTAB	0000000000000000	00002268
	0000000000000000af	0000000000000000		0 0 1
[5]	.shstrtab	STRTAB	0000000000000000	00002317
	000000000000000027	0000000000000000		0 0 1

À 0x402000

À 0x402000 : Un tableau data_table contenant les valeurs attendues après transformation

À 0x402010

À 0x402010 : Une séquence d'indices sequence qui définit l'ordre de vérification des caractères